

# Topic 4.6

## 4.6: delete vs. delete []

When we are finished with a dynamically allocated object, we invoke the `delete` operator on the pointer to the object:

```
SomeType *p;
...
p = new SomeType;    // Creates space for object,
                    // then invokes
                    // SomeType constructor.

...
delete p;            // Invokes SomeType
                    // destructor, then
                    // releases the space.
```

Suppose we had allocated an array of these objects:

```
SomeType *p;
...
p = new    SomeType[5];
          // Creates space for 5 objects, then
          // invokes SomeType constructor
          // on each object.

...
delete p;  // Invokes SomeType destructor
          // on first object only, then releases
          // the space occupied by all 5 objects.
```

If the constructor for `SomeType` dynamically allocates memory or performs some other operation such as opening a file, etc., the destructor is not given an opportunity to undo the operation except for the first object in the dynamically allocated array. This can result in memory leaks or other problems.

In this situation, use the following form:

```
SomeType *p;
...
p = new SomeType[5];
  // Creates space for 5 objects, then
  // invokes SomeType constructor on each object.
...
delete[] p;  // Invokes SomeType destructor
            // on all objects in the array,
            // then releases the space occupied
            // by all 5 objects.
```

This works because the memory management routines keep track of how big the actual allocated block is, and the compiler knows how big each object is. By dividing these two numbers, the number of objects in the array can be determined at runtime, and the destructor is invoked for each.

**[Back to Module 4 \(https://onlinelearning.berkeley.edu/courses/2291660/pages/module-4\)](https://onlinelearning.berkeley.edu/courses/2291660/pages/module-4)**